Monday, October 27, 2025

EXAM POV NOTES (Q&A FORMAT)

2 Marks Questions

Q1. What is the fundamental concept in document databases and their basic structure?

The core component of document databases is the **document** itself. These databases store and retrieve documents, which are defined as self-describing, hierarchical tree data structures.

1. Documents can be stored using common formats such as XML, JSON, or BSON (Binary JSON).
2. They consist of maps, collections, and scalar values.

Q2. How does a document database relate to a traditional key-value store?

Document databases fundamentally rely on the architecture of a key-value store.

1. The database functionality is achieved by storing documents in the **value part** of the underlying key-value store.
2. The documents stored are generally similar to one another but do not have to be exactly the same.

Q3. Define Ad-hoc Queries in the context of MongoDB.

Ad-hoc queries are those queries that were not predefined or known at the time the database schema was initially structured.

1. MongoDB provides support for these types of queries.
2. These queries are updated in real time, which helps lead to an improvement in performance.

Q4. What is the main purpose of Replication in MongoDB?

Replication is the mechanism MongoDB uses to guarantee redundancy and maintain high availability.

1. This feature distributes data to multiple machines.
2. It ensures the system is ready for contingencies, such as when a primary node goes down.

Q5. Explain the function and default size of GridFS.

GridFS is a specific MongoDB feature designed for storing and retrieving files, especially those that exceed 16 MB in size.

1. GridFS achieves this by dividing the large document into parts called chunks.
2. These chunks have a default size of **255kB**, excluding the final chunk.

5 Marks Questions

Q6. Explain why MongoDB is referred to as a Schema-Less database.

MongoDB is considered schema-less because it does not enforce rigid structure restrictions on the data stored, unlike RDBMS.

1. In MongoDB, one collection can hold documents that vary in size, content, and fields.
2. It lacks a strict predefined schema, providing flexibility in dealing with databases.
3. The documents can have differences in attribute names yet still belong to the same collection.
4. The schema of the data is allowed to differ across documents.
5. This flexibility is enabled because the database does not enforce restrictions on the schema.
6. This absence of restriction is a key difference from an RDBMS, where every row must follow the same schema.

Q7. Describe the role of Indexing in improving MongoDB's performance.

Indexing is essential in MongoDB for significantly enhancing the performance and speed of search queries.

1. When continuous searches are performed, fields matching the search criteria should be indexed.
2. MongoDB allows any field to be indexed using both primary and secondary indices.
3. Indexing is directly responsible for making query searches faster.
4. This feature is a major contributor to MongoDB's overall high performance, availability, and scalability.
5. It is a necessary component for fields that are frequently used in query matching.
6. By supporting indexing, MongoDB helps achieve high availability and a faster query response time.

Q8. How does Sharding enable MongoDB to handle large datasets?

Sharding is implemented to manage the complexities and performance issues that arise when dealing with very large datasets in a MongoDB environment.

1. The huge volume of data can cause problems when queries are executed against it.
2. Sharding helps to distribute this problematic data to multiple different MongoDB instances.
3. Collections that reach a larger size are distributed into multiple smaller collections.
4. These resulting distributed collections are specifically referred to as **shards**.
5. Shards are technically implemented by clusters.
6. This distribution allows the database to maintain high availability and scalability even with massive data loads.

Q9. Explain the use of embedding and sub-objects in document database structures.

Document databases allow for rich, hierarchical data structures, which supports embedding related data within a single document.

1. Documents are self-describing tree structures that contain maps, collections, and scalar values.
2. Embedding allows child documents to be stored as **sub objects** inside the main document.
3. This technique can be used to represent a list of addresses as documents embedded inside the main document, or a list of cities as an array.
4. Embedding provides for easy access to related data.
5. It also contributes to achieving better performance.
6. Documents belonging to the same collection are typically similar, but they do not have to be exactly the same.

10 Marks Questions

Q10. Discuss the suitable use cases for Document Databases, such as MongoDB, highlighting their flexibility.

Document databases excel in environments where data needs to evolve quickly and flexibility is crucial, making them ideal for several modern application types.

1. **Event Logging:** They act as a central data store for storing various event types from different enterprise applications, especially when the type of data being captured keeps changing.
2. **Event Sharding:** Events can be effectively sharded by application name or by the specific type of event, such as *order_processed*.
3. **Content Management Systems (CMS):** They are well-suited for CMS applications, publishing websites, and blogging platforms because they inherently understand JSON and lack predefined schemas.
4. **User Data Management:** Suitable for managing user comments, registrations, profiles, and other web-facing documents.
5. **Web Analytics/Real-Time Analytics:** They can store data for real-time analysis.
6. **Flexible Metrics:** New metrics (like page views or unique visitors) can be easily added without requiring schema changes because parts of the document can be updated efficiently.
7. **E-Commerce Applications:** They are highly useful here as e-commerce requires flexible schemas for products and orders, allowing data models to evolve without expensive database refactoring or data migration.
8. **Scaling:** Features like sharding and replication contribute to high availability and scalability, making them suitable for big data and real-time applications.
9. **Application Entities:** Data is saved naturally in the form of application entities (aggregates).
10. **Query Support:** They offer ad-hoc query support, which allows developers to execute queries not known during initial schema design.

These use cases benefit directly from MongoDB's non-rigid, document-oriented structure.

Q11. Identify and explain the specific scenarios where using document databases, like MongoDB, is *not* recommended.

Document databases may be unsuitable when an application requires strict relational integrity, complex transactional logic across multiple data entities, or constant normalization.

1. **Complex Transactions:** They are generally not recommended if there is a need for complex transactions that span multiple different operations.
2. **Atomic Cross-Document Operations:** If the requirement is for atomic operations across multiple documents, document databases may be unsuitable (though some, like RavenDB, offer support).
3. **Varying Aggregate Queries:** Problems occur when queries must be executed against an aggregate structure that is constantly changing.
4. **Ad Hoc Query Changes:** If you need to query entities ad hoc and the join criteria between data structures are continuously changing (analogous to changing table joins in RDBMS).
5. **Normalization Requirement:** They may not work if you need to save aggregates at the **lowest level of granularity**, which requires data normalization.
6. **Data Structure Shifts:** They are problematic if the design of the aggregate is constantly shifting.

7. **Strict Schema Enforcement:** Document databases are unsuitable if strict schema enforcement is required because their flexible schema means the database itself does not enforce any restrictions.
8. **Entity Breakdown:** Since data is saved as application entities, if query requirements regularly break down these aggregates, it indicates a poor fit.
9. **Consistency:** While MongoDB supports consistency, transactions, and availability, requirements for extremely high, traditional RDBMS-style consistency across many entities might lead to choosing a different technology.
10. **Legacy Integrations:** If the core system relies heavily on complex joins and relational integrity checks inherent to SQL, migrating to a document model might be overly complex.

These limitations arise because document databases prioritize flexibility and scalability over strict, enforced relational integrity.